

Universidad Nacional de Costa Rica

Campus Coto

Curso:

EIF207 - Estructuras de Datos

PROYECTO II

Grafos, recorridos y ruta más corta

Docente:

M.Sc. Lenin Chacón Bermúdez

Estudiantes:

Mishia Gabriela Valverde Narváez

María Fernanda Jiménez Alpízar

NRC 90091- Grupo 90

II Ciclo-2026

Descripción del proyecto

El proyecto consiste en desarrollar un planificador de vuelos entre aeropuertos de América. La red se representa mediante un grafo dirigido y ponderado: cada aeropuerto, identificado por su código IATA, corresponde a un vértice, mientras que cada ruta aérea representa una arista dirigida. Cada arista almacena dos pesos independientes: la duración del vuelo en minutos y su costo en dólares. Al ser dirigido, una conexión de A hacia B no implica necesariamente que exista otra de B hacia A.

La aplicación permite consultar los aeropuertos y sus conexiones, encontrar recorridos con la menor cantidad de vuelos, calcular la ruta más rápida y la más económica, determinar qué destinos son alcanzables y analizar qué sucede cuando se cancela una conexión. La información de la red se carga desde archivos de texto, lo que permite modificar los datos sin cambiar la implementación de los algoritmos.

Estructuras de datos utilizadas

La estructura principal es una lista de adyacencia implementada mediante mapas. Un mapa relaciona cada código IATA con el objeto Aeropuerto correspondiente y otro mantiene, para cada aeropuerto, la lista de rutas que salen de él. De esta forma, el programa puede localizar un aeropuerto por su código y recorrer sus conexiones sin revisar todos los pares posibles. Las rutas canceladas se conservan temporalmente para permitir su restauración.

Estructura	Uso en el proyecto
HashMap	Relaciona códigos de aeropuerto con sus datos y listas de rutas salientes.
ArrayList / listas	Almacenan rutas, resultados y conexiones canceladas.
HashSet	Registra los aeropuertos visitados durante BFS y DFS.
ArrayDeque	Funciona como cola para BFS y como pila para DFS.
PriorityQueue	Permite seleccionar eficientemente la distancia menor en Dijkstra.
Mapas de distancias y predecesores	Guardan el costo acumulado y permiten reconstruir las rutas.

Las clases Aeropuerto, Ruta y ResultadoRuta representan la información principal del dominio. La clase Grafo administra los vértices y las conexiones, mientras que GestorRutas coordina la carga de archivos y las operaciones solicitadas por la aplicación. Además, se utilizan listas para almacenar rutas y resultados, conjuntos para controlar los aeropuertos visitados y estructuras auxiliares para ejecutar los algoritmos.

Algoritmos implementados

BFS – ruta con menos escalas

BFS recorre el grafo por niveles utilizando una cola. Primero visita los aeropuertos que están a un vuelo del origen y luego continúa con los siguientes niveles. Cuando encuentra un aeropuerto por primera vez, guarda cuál fue su predecesor; con esa información reconstruye el recorrido al terminar la búsqueda. Una vez obtenida la ruta, se suman los minutos y el costo de sus vuelos. Por esta razón, BFS encuentra el camino con menos vuelos, pero no necesariamente el más rápido ni el más barato.

Complejidad: O(n + m), donde n es la cantidad de aeropuertos y m la cantidad de rutas.

DFS – destinos alcanzables

DFS utiliza una pila para explorar los aeropuertos que pueden alcanzarse desde un origen. El algoritmo marca los vértices visitados y continúa recorriendo sus conexiones hasta que ya no quedan destinos pendientes. Los aeropuertos no alcanzables se identifican comparando los visitados con el conjunto completo de vértices. Para determinar qué aeropuertos pueden llegar a un destino específico, se construye el grafo inverso —con las conexiones en sentido contrario— y se ejecuta DFS desde ese destino.

Complejidad: O(n + m) por recorrido.

Dijkstra – ruta más rápida o más barata

El proyecto utiliza una única implementación de Dijkstra parametrizada por el criterio seleccionado. Si se busca la ruta más rápida, el algoritmo acumula los minutos de cada vuelo; si se busca la más barata, acumula sus costos. Una cola de prioridad permite seleccionar el aeropuerto con la menor distancia conocida, mientras que los mapas de distancias y predecesores almacenan los resultados parciales y permiten reconstruir el camino final. Para garantizar que el resultado sea correcto, los pesos de las rutas deben ser no negativos.

Complejidad: O((n + m) log(n + m)) en la implementación con cola de prioridad. Los pesos deben ser no negativos.

Resultados de las rutas solicitadas

La siguiente tabla resume los resultados obtenidos con los archivos originales del proyecto.

Viaje	Criterio	Ruta	Vuelos	Minutos	Costo
DEN → SJO	Menos escalas	DEN–LAX–SJO	2	500	$895
DEN → SJO	Más rápida	DEN–DFW–IAH–SJO	3	385	$525
DEN → SJO	Más barata	DEN–DFW–SAL–MGA–SJO	4	425	$435
LIR → SJU	Menos escalas	LIR–JFK–SJU	2	510	$585
LIR → SJU	Más rápida	LIR–SJO–PTY–CCS–SJU	4	355	$520
LIR → SJU	Más barata	LIR–SAL–MIA–SJU	3	385	$405
SJO → MVD	Menos escalas	SJO–MIA–BSB–GRU–MVD	4	870	$1125
SJO → MVD	Más rápida	SJO–PTY–LIM–EZE–MVD	4	595	$685
SJO → MVD	Más barata	SJO–PTY–LIM–GRU–MVD	4	720	$640

5. Preguntas de análisis

1. **Para los viajes DEN -> SJO, LIR -> SJU y SJO -> MVD, reporte las tres rutas (menos escalas, más rápida y más barata), cada una con su cantidad de vuelos, minutos y costo. ¿Por qué la ruta de menos escalas no es la más rápida? ¿Por qué la más rápida no es la más barata?**

La tabla del apartado 4 presenta las tres rutas para cada viaje, con su recorrido, cantidad de vuelos, minutos y costo. La ruta con menos escalas no siempre es la más rápida porque BFS minimiza la cantidad de vuelos, no su duración. Asimismo, la ruta más rápida no tiene que ser la más barata, ya que el tiempo y el costo son valores independientes.

2. **¿Por qué BFS no sirve para encontrar la ruta más rápida? ¿Qué condición de los datos garantiza que Dijkstra funcione? ¿Qué pasaría si una ruta tuviera costo negativo (por ejemplo, un reembolso)?**

BFS no encuentra la ruta más rápida porque considera cada vuelo como un paso y minimiza la cantidad de conexiones, sin comparar sus duraciones. Dijkstra requiere pesos no negativos; por eso el programa valida que los minutos y costos no sean negativos. Si se aceptara un costo negativo, como un reembolso, una ruta considerada definitiva podría reducirse después y Dijkstra no garantizaría el resultado correcto.

3. **¿Qué aeropuertos no son alcanzables desde SJO? ¿Desde qué aeropuertos no es posible llegar a SJO? Explique cómo lo calcula su programa.**

Con los archivos originales, KIN no es alcanzable desde SJO. En cambio, ASU y MVD no tienen una ruta que llegue a SJO. El programa obtiene los destinos alcanzables ejecutando DFS desde SJO. Para el sentido inverso, invierte las aristas y vuelve a ejecutar DFS desde SJO; los aeropuertos visitados son aquellos que pueden llegar a ese destino.

4. **Encuentre una ruta cuya cancelación deje algún aeropuerto inalcanzable desde SJO. Explique cómo la encontró. ¿Es la única?**

La ruta LIM → CUZ deja a CUZ inalcanzable desde SJO cuando se cancela. Para encontrarla, se compara el conjunto de destinos alcanzables desde SJO antes y después de cancelar conexiones. En las pruebas con los datos originales, esta fue la única ruta cuya cancelación produjo esa pérdida de alcanzabilidad.

5. **¿Por qué se usa lista de adyacencia y no matriz? Compare la memoria de ambas con estos datos (n = 40, m = 202) y con una red de n = 4 000 aeropuertos con unas 20 rutas cada uno.**

La matriz de adyacencia reserva una posición para cada par ordenado de aeropuertos, aunque no exista una conexión directa entre ellos, por lo que requiere $O(n^2)$ espacio. La lista de adyacencia almacena los vértices y únicamente las rutas existentes, con un espacio de $O(n + m)$.

En los datos del proyecto hay 40 aeropuertos y 202 rutas, por lo que la matriz contempla 1 600 posiciones y la lista alrededor de 242 elementos entre vértices y rutas. En una red estimada de 4 000 aeropuertos y unas 20 rutas salientes por aeropuerto (aproximadamente 80 000 rutas), la diferencia aumenta considerablemente: 16 millones de posiciones para la matriz frente a unos 84 000 elementos para la lista.

6. **Dé la complejidad de BFS, DFS y Dijkstra en su implementación, en términos de n y m. Si usó una cola de prioridad (priority_queue o PriorityQueue), explique qué cambia respecto de buscar el mínimo recorriendo todos los vértices.**

BFS y DFS recorren los vértices y las aristas del grafo un número limitado de veces, por lo que su complejidad es $O(n + m)$, donde n representa la cantidad de aeropuertos y m la cantidad de rutas. Dijkstra utiliza una cola de prioridad para extraer el aeropuerto con la menor distancia conocida. En esta implementación, su complejidad es $O((n + m) \log(n + m))$. Si se buscara el mínimo recorriendo todos los vértices en cada iteración, la selección podría costar $O(n)$ por iteración; la cola de prioridad evita repetir esa búsqueda lineal y hace más eficiente el proceso.

6. Pruebas y validaciones

- Búsqueda de un destino para el que no existe una ruta desde el aeropuerto de origen.
- Cancelación de una conexión, revisión del efecto sobre los recorridos y restauración de la ruta.
- Carga de archivos alternativos para comprobar la construcción del grafo con otros datos.
- Detección de cantidades de registros incorrectas y de líneas faltantes o adicionales.
- Detección de códigos de aeropuerto duplicados y de rutas con extremos inexistentes.
- Rechazo de valores numéricos inválidos, campos incompletos y pesos negativos.

7. Conclusión

El proyecto representa una red aérea mediante un grafo dirigido y ponderado. BFS permite encontrar recorridos con menos vuelos, DFS analiza la conectividad y Dijkstra obtiene rutas de menor tiempo o costo. La lista de adyacencia facilita el manejo de una red dispersa, mientras que las pruebas permiten comprobar el comportamiento del sistema ante rutas inexistentes, cancelaciones y datos inválidos.